

ATIVIDADE PRÁTICA: CLASSIFICAÇÃO DE ESPÉCIES COM O DATASET IRIS

Disciplina: Sistemas de Simulação Computacional | Prof. Ruy Barbosa Figueiredo Junior

04 de maio de 2026

1.

Instituição: Centro Universitário Newton Paiva Wyden
Curso: Engenharia de Computação / Sistemas de Informação
Disciplina: Machine Learning
Docente: Prof. Ruy Barbosa Figueiredo Junior
Data da Atividade: 03 de maio de 2026

2. Objetivos

Esta atividade prática tem como propósito consolidar os conceitos fundamentais de Aprendizado de Máquina Supervisionado por meio da implementação de um fluxo de trabalho (pipeline) completo. Os objetivos específicos incluem:

- 1. Implementar um pipeline de Machine Learning utilizando a biblioteca Scikit-learn.
- 2. Realizar a Análise Exploratória de Dados (EDA) para identificação de padrões multivariados.
- 3. Aplicar técnicas de pré-processamento e divisão de dados em conjuntos de treinamento e teste.
- 4. Treinar e ajustar um modelo de classificação baseado no algoritmo Random Forest (Floresta Aleatória).
- 5. Avaliar o desempenho do modelo utilizando métricas estatísticas de classificação.

3. Contextualização Técnica

O conjunto de dados **Iris**, introduzido pelo estatístico e biólogo britânico Ronald Fisher em 1936, é um dos benchmarks mais utilizados na literatura de reconhecimento de padrões. O dataset é composto por 150 instâncias de flores, distribuídas equitativamente entre três espécies: *Iris setosa*, *Iris versicolor* e *Iris virginica*.

Cada amostra possui quatro atributos preditivos (características) mensurados em centímetros:

Atributo (PT-BR)	Atributo (Original)	Descrição Técnica
Comprimento da Sépala	Sepal Length	Medida longitudinal da sépala da flor.
Largura da Sépala	Sepal Width	Medida transversal da sépala da flor.
Comprimento da Pétala	Petal Length	Medida longitudinal da pétala da flor.
Largura da Pétala	Petal Width	Medida transversal da pétala da flor.

4. Metodologia

A atividade será desenvolvida no ambiente **Google Colab**, utilizando a linguagem Python. A escolha do algoritmo **Random Forest** justifica-se por sua natureza de *Ensemble Learning*, que combina múltiplas árvores de decisão para reduzir a variância e evitar o sobreajuste (overfitting). O processo seguirá as normas de reprodutibilidade científica, utilizando estados aleatórios fixos para garantir que os resultados possam ser validados pelo docente.

5. Parte Prática: Passo a Passo

5.1. Preparação do Ambiente e Carga de Dados

O primeiro passo consiste na importação das bibliotecas essenciais e no carregamento do dataset integrado ao Scikit-learn.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection
import train_test_split from sklearn.ensemble
import RandomForestClassifier from sklearn.metrics
import classification_report, confusion_matrix, accuracy_score

# Carregamento do conjunto de dados
iris = load_iris()

dados_iris = pd.DataFrame(data=iris.data, columns=['comp_sepala', 'larg_sepala', 'comp_petala', 'larg_petala'])
dados_iris['especie'] = iris.target
```

5.2. Divisão dos Dados (Treino e Teste)

Para validar a capacidade de generalização do modelo, dividiremos os dados na proporção de 80% para treinamento e 20% para testes.

```
# Separação de variáveis preditivas (X) e alvo (y)
caracteristicas = dados_iris.drop('especie', axis=1)
alvo = dados_iris['especie']

# Divisão estratificada para manter a proporção das classes
treino_x, teste_x, treino_y, teste_y = train_test_split(
    caracteristicas, alvo, test_size=0.2, random_state=42, stratify=alvo )
```

5.3. Treinamento do Modelo

Instanciaremos o classificador de Floresta Aleatória com 100 estimadores (árvores).

```
# Instanciação e treinamento
modelo_floresta = RandomForestClassifier(n_estimators=100, random_state=42)
modelo_floresta.fit(treino_x, treino_y)

# Realização de previsões com os dados de teste
previsoes = modelo_floresta.predict(teste_x)
```

5.4. Avaliação de Desempenho

A análise final utiliza a matriz de confusão para identificar onde o modelo confundiu as espécies.

Geração de métricas

```
print(f"Acurácia Global: {accuracy_score(teste_y, previsoes):.2f}")
```

```
print(" Relatório de Classificação: ", classification_report(teste_y, previsoes))
```

```
# Visualização da Matriz de Confusão matriz_confusao = confusion_matrix(teste_y, previsoes)
sns.heatmap(matriz_confusao, annot=True, cmap='Blues')
```

```
plt.xlabel('Previsão do Modelo') plt.ylabel('Valor Real') plt.show()
```

6. Exercícios de Fixação

6.1. Questões Fechadas (Múltipla Escolha)

1. Qual a principal vantagem de utilizar o Random Forest em vez de uma única Árvore de Decisão?

- A) Redução do tempo de processamento computacional.
- B) Diminuição do risco de overfitting através da combinação de múltiplos modelos.
- C) Eliminação total da necessidade de pré-processamento de dados.
- D) Capacidade de trabalhar apenas com variáveis categóricas.
- E) Garantia de 100% de acurácia em qualquer dataset.

2. No código apresentado, o parâmetro `random_state=42` serve para:

- A) Aumentar a velocidade de treinamento do modelo.
- B) Definir o número de árvores que serão criadas na floresta.
- C) Garantir a reprodutibilidade dos resultados ao fixar a semente de aleatoriedade.
- D) Selecionar automaticamente as melhores características do dataset.
- E) Normalizar os dados entre 0 e 1.

3. O dataset Iris possui quantas classes de saída (target)?

- A) 2 classes.
- B) 3 classes.
- C) 4 classes.
- D) 150 classes.
- E) 5 classes.

4. A métrica "Acurácia" representa:

- A) A proporção de previsões corretas em relação ao total de casos.
- B) O tempo que o modelo leva para convergir.
- C) A sensibilidade do modelo para a classe minoritária.
- D) A média aritmética das larguras das pétalas.
- E) A distância entre o centroide das classes.

5. O que indica o parâmetro `test_size=0.2` na função `train_test_split`?

- A) Que 2% dos dados serão usados para teste.
- B) Que o modelo será treinado 20 vezes.
- C) Que 20% do conjunto de dados original será reservado para a avaliação final.
- D) Que o erro máximo permitido é de 0.2.
- E) Que o modelo usará apenas 2 características para o teste.

6. Qual biblioteca Python é a principal responsável pelos algoritmos de ML nesta atividade?

- A) Pandas.
- B) Matplotlib.
- C) Scikit-learn.
- D) Seaborn.
- E) Numpy.

7. Na Matriz de Confusão, os elementos na diagonal principal representam:

- A) Os erros do tipo I.
- B) Os erros do tipo II.
- C) As classificações corretas (Verdadeiros Positivos e Negativos).
- D) A importância de cada variável.
- E) O desvio padrão dos dados.

8. O algoritmo Random Forest é classificado como:

- A) Aprendizado Não Supervisionado.
- B) Aprendizado por Reforço.
- C) Aprendizado Supervisionado de Classificação.
- D) Regressão Linear Simples.
- E) Agrupamento (Clustering).

9. Qual atributo do Iris costuma ser o mais determinante para separar a espécie Setosa das demais?

- A) Largura da sépala.
- B) Comprimento da sépala.
- C) Comprimento da pétala.
- D) Cor da pétala.
- E) Peso da flor.

10. O termo "Ensemble Learning" refere-se a:

- A) Limpeza de dados faltantes.
- B) Uso de um único modelo matemático complexo.
- C) Combinação de vários modelos para obter um desempenho preditivo superior.
- D) Visualização de dados em 3D.
- E) Armazenamento de dados em nuvem.

6.2. Questões Abertas (Análise Crítica)

- 1.** Explique a importância da divisão dos dados em conjuntos de treino e teste. O que aconteceria se avaliássemos o modelo usando os mesmos dados do treinamento?
- 2.** Analise o impacto do parâmetro `n_estimators`. Como o aumento do número de árvores afeta a performance e o custo computacional?
- 3.** Por que utilizamos a estratificação (`stratify=alvo`) ao dividir o dataset Iris?
- 4.** Descreva como a Matriz de Confusão auxilia na identificação de gargalos do modelo, especificamente entre as classes *Versicolor* e *Virginica*.
- 5.** O que é *Feature Importance* e como o Random Forest permite identificar quais características da flor são mais relevantes para a classificação?
- 6.** Diferencie as métricas Precisão (Precision) e Revocação (Recall) no contexto deste problema.
- 7.** Se o modelo apresentasse 100% de acurácia no treino e 70% no teste, qual fenômeno estaria ocorrendo? Como resolvê-lo?
- 8.** Como a escala dos dados (cm) influencia o Random Forest? É necessário realizar a normalização ou padronização para este algoritmo específico? Justifique.
- 9.** Discorra sobre a utilidade do F1-Score em datasets onde as classes podem estar desbalanceadas.
- 10.** Proponha uma modificação no pipeline para incluir um novo algoritmo (ex: SVM ou KNN). Quais seriam os passos necessários para comparar os dois modelos de forma justa?